

GNN-FraudNet

AWS EC2 Deployment, Troubleshooting, and Productionization Report

Piyush Maji
MSc Physics, IIT Hyderabad

October 2026

Abstract

This report documents the complete deployment of GNN-FraudNet from a local development environment to a publicly accessible AWS EC2 service. The system combines GraphSAGE, GNNExplainer, graph analysis, ChromaDB-based RAG, LangGraph orchestration, an LLM report generator, FastAPI, Docker, and Nginx.

The deployment exposed several real engineering problems: unreliable SSH connectivity, missing large model/data artifacts, an architecture-mismatched PyTorch wheel, CUDA dependency problems, Python package incompatibilities, Gemini free-tier quota exhaustion, and the need for a secure reverse-proxy architecture. Each issue was diagnosed from concrete build output or runtime logs and resolved without exposing API secrets.

The final service successfully performs the complete investigation workflow through a public dashboard. A representative investigation completed in approximately 3.73 seconds using Groq's `openai/gpt-oss-120b`, with GraphSAGE, GNNExplainer, graph analysis, and ChromaDB RAG functioning end-to-end.

Contents

1	Project Overview	3
2	Deployment Objective	3
3	Final AWS Environment	4
4	Problem 1: SSH Connectivity and Broken Pipes	4
4.1	Observed Problem	4
4.2	Diagnosis	4
4.3	Security	4
5	Problem 2: Missing Large Graph Artifact	4
5.1	Observed Problem	4
5.2	Solution	5
6	Problem 3: Wrong PyTorch CPU Architecture	5
6.1	Observed Problem	5
6.2	Solution	5
7	Problem 4: CUDA Dependency Failure	5
8	Problem 5: MarkupSafe Build Failure	5

9 Problem 6: Transformers / PyTorch Compatibility	6
9.1 Observed Problem	6
9.2 Solution	6
10 Problem 7: Gemini Quota Exhaustion	6
10.1 Observed Problem	6
10.2 Root Cause	6
10.3 Solution	7
11 Problem 8: Groq Integration	7
12 Problem 9: Verifying Groq in Production	7
13 Problem 10: Public Exposure and Nginx	8
14 Problem 11: Security-Group and Port Design	8
15 Runtime Verification	9
16 Representative End-to-End Investigation	9
17 Performance Analysis	10
18 Lessons Learned	10
18.1 Deployment Is an Integration Problem	10
18.2 Pin Critical Dependencies	10
18.3 Separate Failure Layers	10
18.4 Verify Artifacts	11
18.5 External API Abstraction	11
19 Known Limitations and Future Improvements	11
19.1 Smarter Retry Logic	11
19.2 Streaming Provider Support	11
19.3 HTTPS and Domain	11
19.4 Authentication and Rate Limiting	11
19.5 Monitoring	11
20 Final Architecture	11
21 Final Outcome	12

1 Project Overview

GNN-FraudNet is an end-to-end fraud detection and investigation platform. The project moves beyond simple classification by answering three increasingly useful questions:

1. Is this transaction suspicious?
2. Is it suspicious given its graph context?
3. Why is it suspicious, and how should an analyst interpret it?

The Elliptic Bitcoin dataset used by the project contains 203,769 transaction nodes, 234,355 edges, and 166 features per node. The documented labels include 4,545 illicit nodes, 42,019 licit nodes, and 157,205 unknown nodes.

The investigation workflow is:

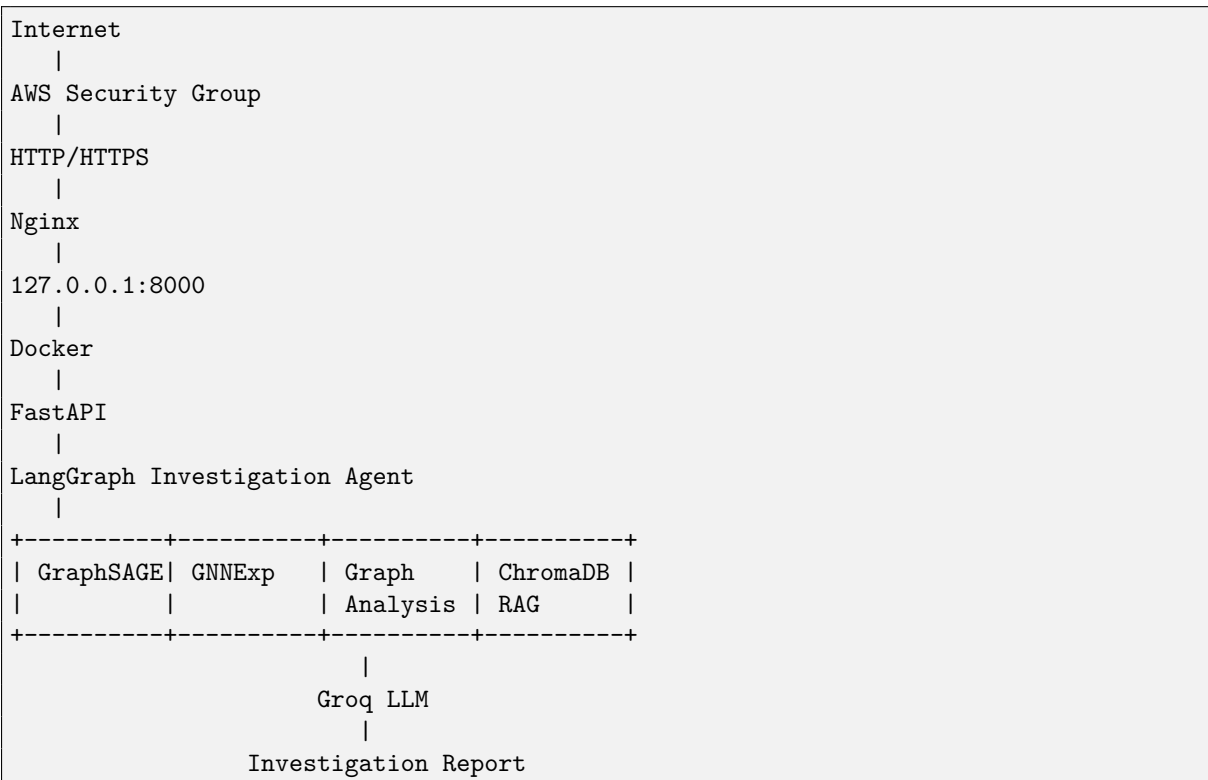
`predict` → `explain` → `analyze_graph` → `retrieve_knowledge` → `generate_report`

The repository separates model inference, explainability, graph analysis, RAG, LLM generation, agent orchestration, and API serving. The documented stack includes PyTorch Geometric, PyTorch, SHAP/GNNEExplainer, ChromaDB, sentence-transformers, LangGraph, FastAPI/Uvicorn, Docker, Docker Compose, and AWS EC2.

2 Deployment Objective

The objective was to make the locally working application available as a real publicly reachable service.

The final architecture is:



A key deployment decision was to bind Docker port 8000 only to localhost:

```
ports:
- "127.0.0.1:8000:8000"
```

Nginx is therefore the public entry point.

3 Final AWS Environment

The actual EC2 deployment used:

- Ubuntu Server 24.04 LTS, x86_64;
- EC2 instance type `m7i-flex.large`;
- 2 vCPUs and 8 GiB RAM;
- approximately 30 GiB root EBS volume;
- Docker and Docker Compose;
- Nginx reverse proxy.

The repository README contains a generic AWS example using a `t3.medium`. The actual deployed machine was `m7i-flex.large`.

SSH port 22 was restricted to the current client IP. Public HTTP was served through port 80, while direct public access to application port 8000 was removed.

4 Problem 1: SSH Connectivity and Broken Pipes

4.1 Observed Problem

AWS Instance Connect was unreliable and direct SSH from the original network experienced connection failures and “Broken pipe” behavior.

4.2 Diagnosis

The EC2 instance was reachable from another network, indicating that the problem was primarily a client-network/path issue rather than an Ubuntu or Docker problem.

The practical solution was to connect through a phone hotspot. SSH keep-alive parameters were also used:

```
ssh -o ServerAliveInterval=30 \  
    -o ServerAliveCountMax=3 \  
    -i ~/Downloads/gnn-key.pem \  
    ubuntu@<EC2_PUBLIC_IP>
```

4.3 Security

The SSH security-group rule was restricted to:

```
TCP 22 -> <CURRENT_CLIENT_IP>/32
```

rather than opening SSH to the entire Internet.

5 Problem 2: Missing Large Graph Artifact

5.1 Observed Problem

The initial Docker build failed because the Dockerfile expected:

```
data/processed/elliptic_graph.pt
```

but the graph artifact was not present on EC2.

5.2 Solution

The graph was transferred with SCP:

```
scp -i ~/Downloads/gnn-key.pem \  
~/Desktop/Project/GNN-FraudNet/data/processed/elliptic_graph.pt \  
ubuntu@<EC2_PUBLIC_IP>:~/GNN-FraudNet/data/processed/
```

The trained model and graph were then checked using SHA-256 hashes. The final hashes matched between local and EC2 copies:

```
fc84d65da12a9428518e3be2722cc2e1ef14649a4f26b33d7fe3d259e0fdafce  results/  
best_graphsage.pt  
e05d1ca6a69e324f7ce5ebace75041df7746fa6b8115e5719fcf0a8411dbcdd3  data/processed/  
elliptic_graph.pt
```

This verified that production used the intended model and graph data.

6 Problem 3: Wrong PyTorch CPU Architecture

6.1 Observed Problem

The Docker wheel cache initially contained an ARM64 PyTorch wheel, while EC2 was x86_64.

6.2 Solution

The incorrect wheel was replaced by an x86_64 CPU-only PyTorch 2.4.0 wheel:

```
docker run --rm \  
-v "$PWD/docker_wheels:/wheels" \  
python:3.11-slim \  
pip download torch==2.4.0 \  
--index-url https://download.pytorch.org/whl/cpu \  
--python-version 311 \  
--platform linux_x86_64 \  
--only-binary=:all: \  
--no-deps \  
-d /wheels
```

The CPU wheel was approximately 187 MB.

7 Problem 4: CUDA Dependency Failure

The first x86_64 wheel was CUDA-enabled. Because the Docker build intentionally installed from a local wheel directory using `-no-index`, CUDA dependencies could not be resolved.

The solution was to use the CPU-only PyTorch wheel. Since EC2 was performing inference rather than GPU training, CUDA was unnecessary.

This simplified the runtime dependency graph:

PyTorch CPU instead of PyTorch + CUDA runtime stack.

8 Problem 5: MarkupSafe Build Failure

During dependency installation, the Docker build encountered a failure involving `MarkupSafe>=2.0`. A compatible MarkupSafe wheel was added to the local Docker wheel cache so installation could occur without an unwanted source build.

After this adjustment, the Docker image built successfully.

9 Problem 6: Transformers / PyTorch Compatibility

9.1 Observed Problem

After the first successful image build, startup logs reported:

```
[transformers] Disabling PyTorch because PyTorch >= 2.5
is required but found 2.4.0+cpu
NameError: name 'nn' is not defined
```

The issue was a compatibility mismatch between the pinned PyTorch 2.4.0 environment and newer installed versions of Transformers/Sentence Transformers.

9.2 Solution

The Docker requirements were pinned:

```
sentence-transformers==3.3.1
transformers==4.46.3
```

After a no-cache rebuild, the health endpoint reported:

```
{
  "status": "ok",
  "model": "GraphSAGE",
  "rag_available": true,
  "llm_available": true,
  "investigation_available": true
}
```

This confirmed that model inference, RAG, LLM integration, and the investigation agent were all available.

10 Problem 7: Gemini Quota Exhaustion

10.1 Observed Problem

The original LLM provider was Google Gemini. The application successfully reached Gemini, but the free-tier request quota was eventually exhausted.

The logs showed:

```
HTTP ... 429 Too Many Requests
RESOURCE_EXHAUSTED
Quota exceeded metric ... generate_content_free_tier_requests
```

The existing retry logic waited:

1 s, 2 s, 4 s

between attempts. Consequently, quota failures added roughly seven seconds of delay.

10.2 Root Cause

The GNN, RAG, Docker, and FastAPI pipeline was not the source of the failure. The external LLM provider had exhausted its quota.

10.3 Solution

Instead of removing Gemini, the application was changed to support selectable LLM providers:

```
GEMINI_API_KEY
GROQ_API_KEY
LLM_PROVIDER
LLM_MODEL
```

The active deployment uses:

```
LLM_PROVIDER=groq
LLM_MODEL=openai/gpt-oss-120b
```

Gemini remains available as an alternative provider.

11 Problem 8: Groq Integration

The report generator was modified so client initialization depends on the selected provider:

```
if LLM_PROVIDER == "groq":
    client = Groq(api_key=GROQ_API_KEY)
else:
    client = Gemini(api_key=GEMINI_API_KEY)
```

The Groq request uses the OpenAI-compatible interface:

```
response = client.chat.completions.create(
    model=self.model,
    messages=[{"role": "user", "content": prompt}],
    temperature=LLM_TEMPERATURE,
    max_tokens=LLM_MAX_TOKENS,
)
```

The Groq dependency was added:

```
groq>=0.30.0
```

API keys were stored only in `.env`. The repository already ignored `.env`, and it was also added to `.dockerignore`. Docker Compose passes the environment into the running container without copying the secret into the image.

12 Problem 9: Verifying Groq in Production

The rebuilt container was recreated:

```
docker compose build
docker compose up -d --force-recreate
docker compose ps
```

The service became healthy.

A full investigation was then executed:

```
time curl -s -X POST \
http://<PUBLIC_IP>/investigate/78642 > groq.json
```

Measured time:

3.739 seconds

The application logs explicitly confirmed:

```
HTTP Request:
POST https://api.groq.com/openai/v1/chat/completions
HTTP/1.1 200 OK
```

followed by:

```
Step 5/5 [LLM]: Report generated (3061 chars)
Investigation complete for node 78642 in 3.73s
```

Thus Groq was verified as an actual production dependency, not merely a configuration value.

13 Problem 10: Public Exposure and Nginx

FastAPI/Uvicorn listens internally on port 8000. Instead of exposing this port publicly, Nginx was installed as a reverse proxy.

The effective configuration was:

```
server {
    listen 80;
    server_name <PUBLIC_IP>;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

The configuration was validated using:

```
sudo nginx -t
```

The default site was disabled and Nginx reloaded.

The final request path is:

```
Browser -> AWS Security Group -> Nginx :80
        -> 127.0.0.1:8000 -> Docker -> FastAPI
```

14 Problem 11: Security-Group and Port Design

The final Docker binding is:

```
127.0.0.1:8000:8000
```

Therefore, port 8000 is not directly reachable from the Internet.

The security group was configured conceptually as:

- SSH 22: restricted to the current client IP;
- HTTP 80: public;
- HTTPS 443: available;
- direct public port 8000: removed.

This produces a cleaner separation between the public web layer and the application process.

15 Runtime Verification

At startup, the container logs showed:

```
Loaded graph: 203769 nodes, 234355 edges
Loaded existing knowledge base: 43 chunks
RAG knowledge base initialized
LLM report generator initialized
Investigation agent initialized
Application startup complete
```

The health endpoint reported all major components available:

```
{
  "status": "ok",
  "model": "GraphSAGE",
  "rag_available": true,
  "llm_available": true,
  "investigation_available": true
}
```

16 Representative End-to-End Investigation

Node 78642 was used as a deployment verification case.

The final prediction was:

- fraud probability: 0.1628465;
- licit probability: 0.8371535;
- label: `licit`;
- confidence: `high`.

Graph analysis reported:

- in-degree: 1;
- out-degree: 1;
- total degree: 2;
- one-hop illicit neighbors: 0;
- two-hop illicit neighbors: 0;
- structural flag: `mostly_unknown_neighbors`.

GNNEExplainer produced ten important features. The highest-ranked were:

Rank	Feature	Importance
1	feature_149	0.8735
2	feature_161	0.8727
3	feature_7	0.8507
4	feature_5	0.8410
5	feature_64	0.8193
6	feature_116	0.7794

7	feature_158	0.6820
8	feature_22	0.6509
9	feature_148	0.6390
10	feature_54	0.6360

The RAG system retrieved four relevant knowledge chunks, and the Groq LLM converted the structured evidence into a natural-language investigation report.

17 Performance Analysis

The deployment exposed the difference between local computation and external inference.

The GNN prediction, explanation, graph analysis, and RAG retrieval were all sub-second operations. The external LLM request dominated the successful end-to-end latency.

The final Groq-backed investigation completed in:

3.73 s

The earlier Gemini quota-failure path took approximately 8.5 seconds because the application retried three times after receiving HTTP 429 responses.

Therefore, the major performance bottleneck is the external LLM call, not the GNN or RAG pipeline.

18 Lessons Learned

18.1 Deployment Is an Integration Problem

The deployment required compatibility across:

PyTorch+PyG+Transformers+Sentence Transformers+ChromaDB+LangGraph+FastAPI+Docker+AWS+LLM

A failure in one layer can prevent the entire service from starting.

18.2 Pin Critical Dependencies

The Transformers/Sentence Transformers problem demonstrated why production environments should pin critical package versions rather than relying on arbitrary latest versions.

18.3 Separate Failure Layers

Failures occurred at several independent layers:

1. client network/SSH;
2. AWS configuration;
3. Docker build;
4. Python dependencies;
5. application initialization;
6. external API quota;
7. reverse proxy/public networking.

Classifying failures by layer made diagnosis much faster.

18.4 Verify Artifacts

SHA-256 checksums provided strong evidence that the production server was using exactly the intended model and graph artifacts.

18.5 External API Abstraction

The provider switch demonstrated the value of an LLM abstraction:

$$\text{Application} \rightarrow \text{LLM Interface} \rightarrow \{\text{Gemini, Groq}, \dots\}.$$

This is more resilient than hard-coding a single provider.

19 Known Limitations and Future Improvements

19.1 Smarter Retry Logic

The application should distinguish transient rate limits from daily quota exhaustion. A permanent quota error should not trigger several immediate retries.

19.2 Streaming Provider Support

The main non-streaming investigation path was migrated to Groq. The separate streaming report path was originally Gemini-specific and should be updated if streaming is required.

19.3 HTTPS and Domain

A proper domain plus TLS certificate would make the public deployment more polished:

$$\text{Domain} \rightarrow \text{Nginx} \rightarrow \text{TLS} \rightarrow \text{FastAPI}.$$

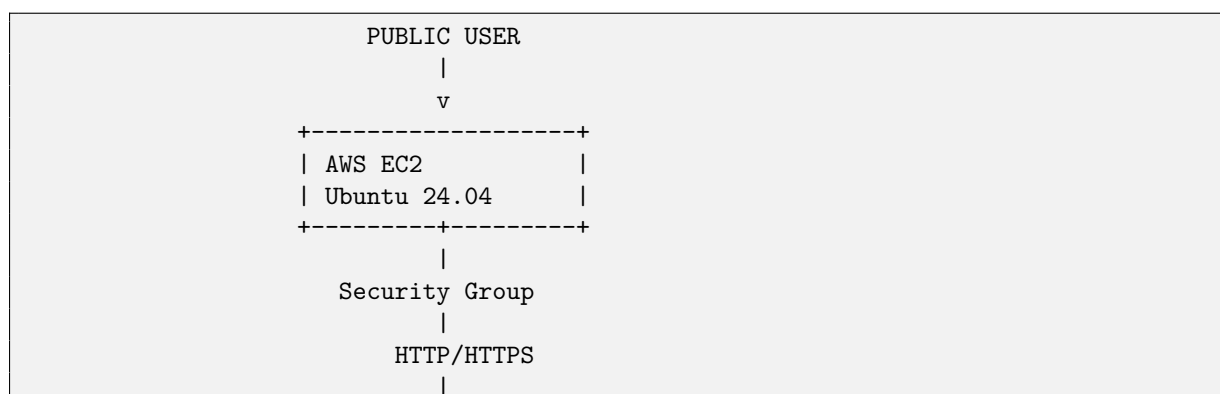
19.4 Authentication and Rate Limiting

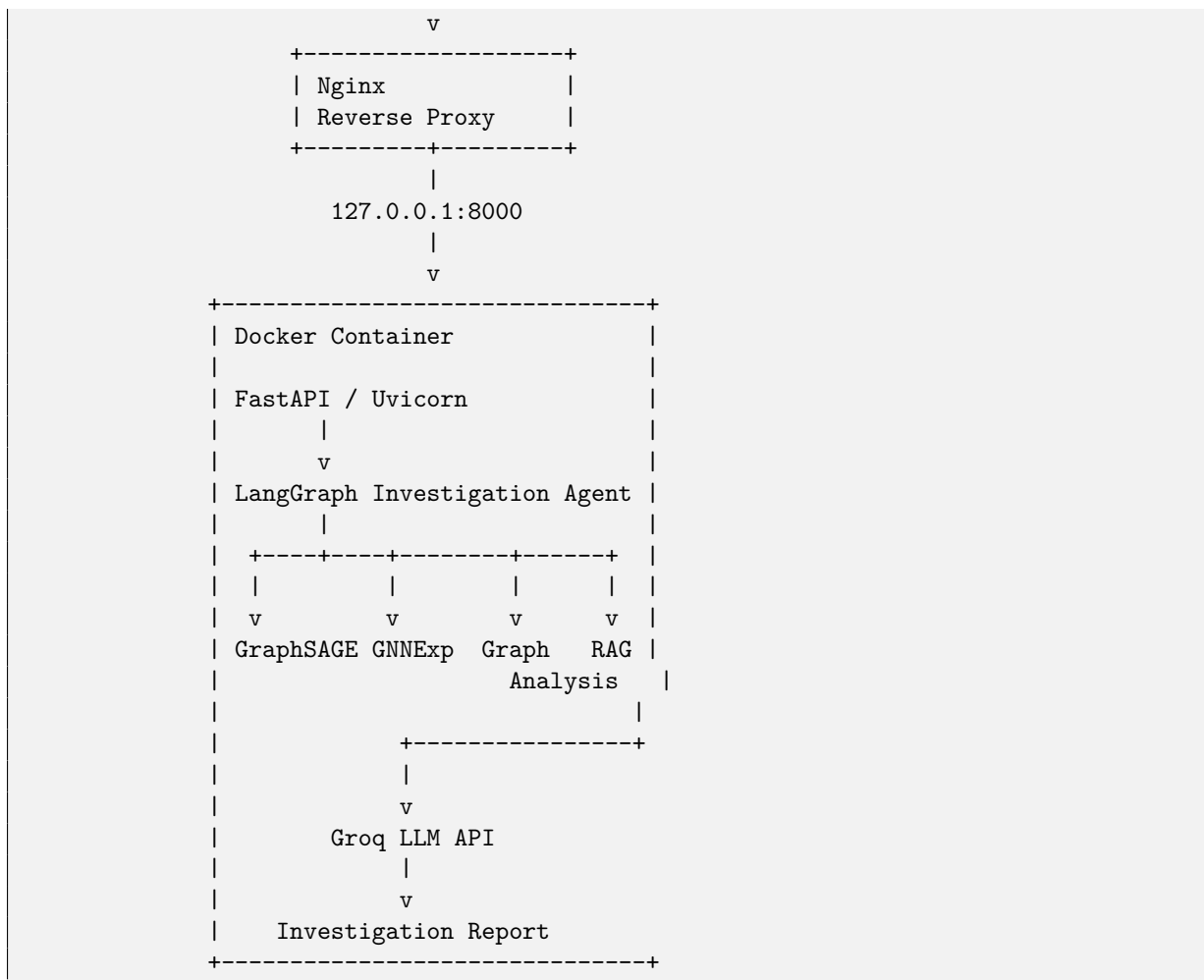
A production financial application should add authentication, rate limiting, abuse protection, secret rotation, and appropriate CORS restrictions.

19.5 Monitoring

Useful production metrics include request latency, model inference latency, explanation latency, RAG latency, LLM latency, error rates, CPU/memory utilization, and container health.

20 Final Architecture





21 Final Outcome

The final production path was verified as:

Browser → Nginx → FastAPI → LangGraph → GraphSAGE → GNNExplainer → Graph Analysis → Chroma

The public dashboard was subsequently tested through the browser and confirmed to work.

The key engineering result is that the complete AI investigation system was made containerized, publicly accessible, operational, and resilient to a change in LLM provider while keeping the trained model and graph artifacts intact.

Deployment Checklist

Item	Status
AWS EC2 provisioned	✓
SSH access established	✓
Docker installed	✓
Docker Compose installed	✓
Model transferred and checksum verified	✓
Processed graph transferred and checksum verified	✓

CPU-compatible PyTorch installed	✓
Dependency compatibility fixed	✓
RAG knowledge base initialized	✓
FastAPI application healthy	✓
Nginx reverse proxy configured	✓
Direct public port 8000 removed	✓
Gemini provider retained as option	✓
Groq provider integrated	✓
Groq API request verified with HTTP 200	✓
End-to-end investigation verified	✓
Public dashboard verified	✓
